SQL QUERY IMPLEMENTATION FOR E-COMMERCE DATABASE

Project Title:

Bookswagon – Online Book Store Order Management Database System

SUBQUERIES AND NESTED QUERIES

A subquery is a query placed inside another query. The inner query calculates a value, and the outer query uses that value to filter results.

1 . Orders Above Average Order Value

```
SQL> SELECT Order_ID, Customer_ID, Total_Amount
   2  FROM Orders
   3  WHERE Total_Amount > (SELECT AVG(Total_Amount) FROM Orders)
   4  ORDER BY Total_Amount DESC;

ORDER_ID  CUSTOMER_ID  TOTAL_AMOUNT
--------  -----------  ------------
    1008            8       1297.00
    1002            2       1149.00
    1007            7       1149.00
    1005            5       1049.00

4 rows selected.
```

2. Books Priced Above Average

```
SQL> SELECT Book_ID, Title, Price
   2  FROM Book
   3  WHERE Price > (SELECT AVG(Price) FROM Book)
   4  ORDER BY Price DESC;

BOOK_ID  TITLE                     PRICE
-------  ------------------------  ------
    112  Fundamentals of DBMS      950.00
    103  Database System Concepts  850.00
    114  Ponniyin Selvan           699.00
    110  Harry Potter Stone        599.00
    102  Atomic Habits             499.00

5 rows selected.
```

Task 3 – Books Priced Below Average

```
SQL> SELECT Book_ID, Title, Price
   2  FROM Book
   3  WHERE Price < (SELECT AVG(Price) FROM Book)
   4  ORDER BY Price ASC;

BOOK_ID  TITLE                 PRICE
-------  --------------------  ------
    104  Panchatantra Stories  199.00
    113  The Jungle Book       225.00
    107  Think and Grow Rich   275.00
    101  The Alchemist         299.00
    109  Wings of Fire         325.00
    106  The Kite Runner       350.00
    108  Let Us C              450.00

7 rows selected.
```

.

4. Step 1: Total Purchase per Customer

```
SQL> SELECT Customer_ID, SUM(Total_Amount) AS Total_Purchase
   2  FROM Orders
   3  GROUP BY Customer_ID
   4  ORDER BY Customer_ID;

CUSTOMER_ID  TOTAL_PURCHASE
-----------  --------------
          1          873.00
          2         1149.00
          3          850.00
          4          947.00
          5         1049.00
          6          950.00
          7         1149.00
          8         1297.00
          9          550.00

9 rows selected.
```

5. Step 2: Maximum Total Purchase

```
SQL> SELECT MAX(Total_Purchase) AS Max_Purchase
   2  FROM (SELECT Customer_ID, SUM(Total_Amount) AS Total_Purchase
   3        FROM Orders
   4        GROUP BY Customer_ID) t;

MAX_PURCHASE
------------
     1297.00

1 rows selected.
```

6. Step 3: Customer with Maximum Purchase

```
SQL> SELECT c.Customer_ID, c.Customer_Name, SUM(o.Total_Amount) AS Total_Purchase
   2  FROM Customer c
   3  JOIN Orders o ON c.Customer_ID = o.Customer_ID
   4  GROUP BY c.Customer_ID, c.Customer_Name
   5  HAVING SUM(o.Total_Amount) = (SELECT MAX(Total_Purchase)
   6                                FROM (SELECT Customer_ID, SUM(Total_Amount) AS Total_Purchase
   7                                      FROM Orders
   8                                      GROUP BY Customer_ID) t);

CUSTOMER_ID  CUSTOMER_NAME  TOTAL_PURCHASE
-----------  -------------  --------------
          8  Meena                1297.00

1 rows selected.
```

COMPLEX BUSINESS QUERIES

7. Customers with Order Value Above Average Order Value

```
SQL> SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Total_Amount
   2  FROM Customer c
   3  JOIN Orders o ON c.Customer_ID = o.Customer_ID
   4  WHERE o.Total_Amount > (SELECT AVG(Total_Amount) FROM Orders)
   5  ORDER BY o.Total_Amount DESC;

CUSTOMER_ID  CUSTOMER_NAME  ORDER_ID  TOTAL_AMOUNT
-----------  -------------  --------  ------------
          8  Meena              1008       1297.00
          2  Hema               1002       1149.00
          7  Arun               1007       1149.00
          5  Priya              1005       1049.00

4 rows selected.
```

8. Books Whose Sales Are Above Average

Sales of a book are calculated from the Order_Item table as the sum of Total_Amount (Quantity × Price). The subquery finds the average sales across all books.

```
SQL> SELECT b.Book_ID, b.Title, SUM(oi.Total_Amount) AS Total_Sales
   2  FROM Book b
   3  JOIN Order_Item oi ON b.Book_ID = oi.Book_ID
   4  GROUP BY b.Book_ID, b.Title
   5  HAVING SUM(oi.Total_Amount) > (SELECT AVG(Book_Sales)
   6                                 FROM (SELECT Book_ID, SUM(Total_Amount) AS Book_Sales
   7                                       FROM Order_Item
   8                                       GROUP BY Book_ID) t)
   9  ORDER BY Total_Sales DESC;

BOOK_ID  TITLE                    TOTAL_SALES
-------  ------------------------  -----------
    102  Atomic Habits                1497.00
    112  Fundamentals of DBMS          950.00
    101  The Alchemist                 897.00
    103  Database System Concepts      850.00
    107  Think and Grow Rich           825.00

5 rows selected.
```

ADVANCED SALES REPORTS

9. Customer Purchase Report

```
SQL> SELECT c.Customer_ID, c.Customer_Name,
   2         COUNT(o.Order_ID) AS Orders,
   3         SUM(o.Total_Amount) AS Total_Purchase,
   4         ROUND(AVG(o.Total_Amount), 2) AS Avg_Order_Value
   5  FROM Customer c
   6  JOIN Orders o ON c.Customer_ID = o.Customer_ID
   7  GROUP BY c.Customer_ID, c.Customer_Name
   8  ORDER BY c.Customer_ID;

CUSTOMER_ID  CUSTOMER_NAME  ORDERS  TOTAL_PURCHASE  AVG_ORDER_VALUE
-----------  -------------  ------  --------------  ---------------
          1  Nivethitha          1          873.00           873.00
          2  Hema                1         1149.00          1149.00
          3  Ravi                1          850.00           850.00
          4  Karthik             1          947.00           947.00
          5  Priya               1         1049.00          1049.00
          6  Divya               1          950.00           950.00
          7  Arun                1         1149.00          1149.00
          8  Meena               1         1297.00          1297.00
          9  Nisha               1          550.00           550.00

9 rows selected.
```

10. Advanced Book Sales Report

```
SQL> SELECT b.Book_ID, b.Title,
   2         COUNT(DISTINCT oi.Order_ID) AS Orders,
   3         SUM(oi.Quantity) AS Qty_Sold,
   4         SUM(oi.Total_Amount) AS Total_Sales,
   5         ROUND(AVG(oi.Price), 2) AS Avg_Price
   6  FROM Book b
   7  JOIN Order_Item oi ON b.Book_ID = oi.Book_ID
   8  GROUP BY b.Book_ID, b.Title
   9  ORDER BY Total_Sales DESC;

BOOK_ID  TITLE                     ORDERS  QTY_SOLD  TOTAL_SALES  AVG_PRICE
-------  ------------------------  ------  --------  -----------  ---------
    102  Atomic Habits                  2         3      1497.00     499.00
    112  Fundamentals of DBMS           1         1       950.00     950.00
    101  The Alchemist                  2         3       897.00     299.00
    103  Database System Concepts       1         1       850.00     850.00
    107  Think and Grow Rich            2         3       825.00     275.00
    114  Ponniyin Selvan                1         1       699.00     699.00
    109  Wings of Fire                  1         2       650.00     325.00
    110  Harry Potter Stone             1         1       599.00     599.00
    104  Panchatantra Stories           1         3       597.00     199.00
    108  Let Us C                       1         1       450.00     450.00
    113  The Jungle Book                1         2       450.00     225.00
    106  The Kite Runner                1         1       350.00     350.00

12 rows selected.
```

11. Category-wise Advanced Sales Report

```
SQL> SELECT cat.Category_ID, cat.Category_Name,
   2         COUNT(DISTINCT oi.Order_ID) AS Orders,
   3         SUM(oi.Quantity) AS Qty_Sold,
   4         SUM(oi.Total_Amount) AS Total_Sales,
   5         ROUND(AVG(oi.Price), 2) AS Avg_Price
   6  FROM Category cat
   7  JOIN Book b ON cat.Category_ID = b.Category_ID
   8  JOIN Order_Item oi ON b.Book_ID = oi.Book_ID
   9  GROUP BY cat.Category_ID, cat.Category_Name
  10  ORDER BY Total_Sales DESC;

CATEGORY_ID  CATEGORY_NAME  ORDERS  QTY_SOLD  TOTAL_SALES  AVG_PRICE
-----------  -------------  ------  --------  -----------  ---------
          2  Self-Help           4         8      2972.00     374.60
          1  Fiction             5         6      2545.00     449.20
          3  Academic            3         3      2250.00     750.00
          4  Children            2         5      1047.00     212.00

4 rows selected.
```